



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Λειτουργικά συστήματα
Εργασία 2

Ονοματεπώνυμο : Ζουμπουρλής Θοδωρής

Αριθμός Μητρώου : 1115202200267

1 Στόχος

Στόχος της εργασίας ήταν η αντικατάσταση του βασικού round-robin scheduler του xv6 με έναν **MLFQ (Multilevel Feedback Queue) scheduler** με 4 επίπεδα προτεραιότητας και η υλοποίηση μιας νέας κλήσης συστήματος `getpinfo` που επιτρέπει την ανάκτηση πληροφοριών για τις ενεργές διεργασίες. Επιπλέον, υλοποιήθηκε πρόγραμμα χρήστη `ps` που εμφανίζει τα δεδομένα αυτά.

2 Προσέγγιση Υλοποίησης

2.1 Δομή

Η δομή `struct proc` επεκτάθηκε ώστε να περιλαμβάνει τα νέα πεδία που απαιτούνται από τον MLFQ scheduler:

- `int priority`: Προτεραιότητα της διεργασίας (0–3, 0 = υψηλότερη).
- `int ticks_consumed`: Ticks που έχει καταναλώσει η διεργασία στο τρέχον επίπεδο.
- `int wait_ticks`: Ticks που η διεργασία περιμένει χωρίς να εκτελείται (για starvation boost).

2.2 Scheduler

Ο scheduler τροποποιήθηκε για να υποστηρίζει **MLFQ**:

1. Δημιουργήθηκαν 4 επίπεδα προτεραιότητας (0–3).
2. Η αναζήτηση για εκτελέσιμη διεργασία ξεκινά πάντα από την υψηλότερη προτεραιότητα.
3. Μέσα σε κάθε επίπεδο, οι διεργασίες εκτελούνται με *round-robin*.
4. Με κάθε timer tick:
 - Αυξάνεται ο μετρητής `ticks_consumed` για τη διεργασία που εκτελείται.

- Αν η διεργασία καταναλώσει όλο το χρονομερίδιο του επιπέδου της, υποβιβάζεται στο επόμενο επίπεδο και μηδενίζεται το `ticks_consumed`.
 - Οι διεργασίες που είναι `RUNNABLE` αλλά δεν εκτελούνται αυξάνουν τον `wait_ticks`. Αν φτάσει 10 φορές το χρονομερίδιο, γίνεται **boost προτεραιότητας** για αποφυγή starvation.
5. Ο μηχανισμός `goto start_over` επιτρέπει στον scheduler να ελέγχει ξανά από την υψηλότερη ουρά μόλις ολοκληρωθεί η εκτέλεση μιας διεργασίας.

Τα χρονομερίδια για κάθε επίπεδο ορίζονται στον πίνακα:

$$\text{quota}[] = \{4, 8, 16, 32\}$$

2.3 Κλήση Συστήματος `getpinfo`

1. Προστέθηκε νέα δομή `struct pstat` που περιέχει πίνακες για `pid`, `priority`, `state`, `name` και `inuse` για κάθε διεργασία.
2. Η συνάρτηση πυρήνα `collect_pinfo(struct pstat *ps)` συλλέγει τα δεδομένα από τον πίνακα `proc`.
3. Η κλήση συστήματος `sys_getpinfo` αντιγράφει τα δεδομένα στο user space.
4. Προστέθηκε αριθμός κλήσης συστήματος `SYS_getpinfo = 22` στο `syscall.h`, δηλώσεις στο `user.h` και εγγραφή στην `usys.pl`.

2.4 Πρόγραμμα Χρήστη `ps`

Το πρόγραμμα `ps.c` καλεί την `getpinfo` και εμφανίζει:

- PID κάθε διεργασίας
- Κατάσταση (`RUNNING`, `SLEEPING`, `RUNNABLE`, `ZOMBIE`)
- Επίπεδο προτεραιότητας
- Όνομα διεργασίας

2.5 Διαχείριση Ticks και Παράδοση CPU

- Στο `trap.c` και `kerneltrap()` κάθε timer interrupt καλεί `update_proc_stats()` για ενημέρωση του MLFQ.
- Αν η τρέχουσα διεργασία έχει τελειώσει το χρονομερίδιό της, καλείται για να παραδώσει τη CPU.

3 Προβλήματα και Λύσεις

- **Συνθήκες race:** Χρήση του `p->lock` για ασφαλή πρόσβαση στη δομή `proc`.
- **Starvation:** Προστέθηκε μηχανισμός προτεραιότητας για διεργασίες που περιμένουν πολύ.
- **Timer ticks και χρονομερίδια:** Διακριτός χειρισμός των ticks που καταναλώνονται και των συνολικών χρόνων αναμονής.

4 Συμπέρασμα

Η υλοποίηση του MLFQ scheduler στο xv6 επιτρέπει:

- Προτεραιοποίηση διεργασιών με διαφορετικά χρονομερίδια.
- Ελαχιστοποίηση starvation μέσω boost προτεραιότητας.
- Παρακολούθηση και εμφάνιση στατιστικών διεργασιών με `getpinfo`.